

Advanced Testing Techniques

Robert Bocchino, Kevin F. Ortega
NASA Jet Propulsion Laboratory
February 13, 2026

Copyright © 2026 California Institute of Technology.
Government sponsorship acknowledged.

Jet Propulsion Laboratory
California Institute of Technology

Introduction

- Thorough testing is a key part of FSW development
- This course assumes basic familiarity with testing in F Prime
  - Unit testing of components
  - Integration testing of deployments
- This section will cover advanced testing techniques
  - Unit testing with rules and scenarios
  - Reusable functional integration testing (FIT)
  - Continuous integration and continuous deployment

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Traditional Unit Testing

- Approach
  - For each requirement, write one or more tests by hand
  - Review code coverage, add tests
  - Iterate to get to 100% coverage
- Issues with this approach
  - Significant manual effort, e.g., writing sequences of commands
  - Even with 100% code coverage, many behaviors may be unchecked
  - Without careful coding, tests can become unmaintainable

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Another Way: Scenario-Based Unit Testing

- Organize tests as **scenarios** that apply **rules**
- **Rule:** A simple, reusable test behavior consisting of
  - *A precondition (predicate on state):* When can this rule be applied?
  - *An action:* Call the interface under test, check the result
    - Example: Issue a command, check for expected telemetry
- **Scenario:** A recipe for using rules to construct tests. Examples:
  - Fixed sequence, random sequence
  - Pre-programmed simulation, e.g., simulate a duty cycle
  - Composition of scenarios, e.g., interleaved, condition, loop

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Example: Rules for Testing a Buffer Manager

Rule	Precondition	Action
Allocate_OK	A buffer is available	Request a buffer; check that the request succeeds
Deallocate_OK	At least one buffer is allocated	Deallocate a buffer; check that the operation succeeds
Allocate_NoBuffers	All buffers are allocated	Request a buffer; check that the request fails with the expected error

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Example: Sequence Scenarios

Allocate.OK	Deallocate.OK	Allocate.NoBuffers
Apply rule *Allocate_OK*	Apply rule *Allocate_OK* Apply rule *Deallocate_OK*	Do *maxNumBuffers* times: Apply rule *Allocate_OK* Apply rule *Allocate_NoBuffers*

Initial state: No buffers are allocated

Apply: Check the precondition and run the action

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.



Example: Random Scenario

[Scenario Selected.Random] Applying rule Deallocate.NoAllocations
[Scenario Selected.Random] Applying rule Allocate.OK
[Scenario Selected.Random] Applying rule Allocate.BufferTooLarge
[Scenario Selected.Random] Applying rule Allocate.OK
[Scenario Selected.Random] Applying rule Allocate.BufferTooLarge
[Scenario Selected.Random] Applying rule Allocate.BufferTooLarge
[Scenario Selected.Random] Applying rule Deallocate.OK
[Scenario Selected.Random] Applying rule Deallocate.OK
[Scenario Selected.Random] Applying rule Deallocate.NoAllocations
...

Random selector picks only rules whose precondition is true

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Benefits

- Well-structured tests
  - Rules and scenarios provide an easy-to-follow pattern
  - Pattern wards off bad coding, e.g., copy-paste
  - Tests are easy to understand and maintain
- High-quality tests
  - Rules model the system external to the module under test
  - Tests are automatically generated from scenarios
    - Can apply many rules (thousands or millions)
    - Cover many more behaviors than manual testing
    - Can use model checkers or fuzz testers to pick rules

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

STest

- An open-source C++ framework for scenario-based testing
  - Available at https://github.com/fprime-community/stest
  - Included in the F Prime repository at *fprime/STest*
- Rule is a class template
  - State type is a template parameter
  - Precondition and action are pure virtual functions
- Scenarios are a type hierarchy of class templates
- Includes a generic interface called *Pick* for picking input values
  - Default is pseudo-random, with seed capture for replay
  - Can also use Spin model checker

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Random Testing with STest

- Call *Stest::Random::seed()* to seed the random number generator

```
#include <gtest/gtest.h>

#include "STest/Random/Random.hpp"

int main(int argc, char** argv) {
    ::testing::InitGoogleTest(&argc, argv);
    STest::Random::seed();
    return RUN_ALL_TESTS();
}
```

- Use files in the unit test directory to control and monitor behavior
  - Run *touch show-rules* to show rule applications, *rm show-rules* to revert
  - Run *cat seed-history* to see the history of random seeds used
  - Run *tail –n 1 seed-history > seed* to capture and replay the last seed value
- Unit test output reports the seed value used. For example:
  - **Seed file does not exist:** [STest::Random] Generated seed 739331 from system time
  - **Seed file exists:** [STest::Random] Read seed 739331 from file

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

F Prime Modules with Scenario Tests

Build Module	Scenario Tests Used For
Fw/DataStructures	Testing the data structure library
Fw/Logger	Testing the logger
Os	Testing various capabilities in the OS abstraction layer
Svc/Ccsds/ApidManager	Testing the CCSDS APID manager
Svc/DpManager	Testing the data product manager
Svc/DpWriter	Testing the data product writer
Utils/Types	Testing the serial circular buffer

You can look at these modules for real-world examples

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Reusable FIT

- Functional Integration Testing (FIT)
  - Python-based API that uses pytest
    - A custom fixture called fprime_test_api is provided that configures pytest to attach to the F Prime GDS

```
def test_send_command(fprime_test_api):
    """Ensure commanding works as expected
    Tests that commands may be sent and the expected command severity events are returned.
    """

    fprime_test_api.send_and_assert_command(
        fprime_test_api.get_mnemonic("Svc.CommandDispatcher") + "." + "CMD_NO_OP",
        max_delay=1,
    )

    assert fprime_test_api.get_command_test_history().size() == 1
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Reusable FIT

- Component FIT script
  - MyComponent/test/int/<fit-script>.py
  - Deployment independent by using ***`fprime_test_api.get_mnemonic`***
  - Behavior tests -- supplemental to unit tests

- Deployment FIT script
  - MyDeployment/test/int/<fit-script>.py
  - Define integration and test configuration at MyDeployment/test/int/int_config.json

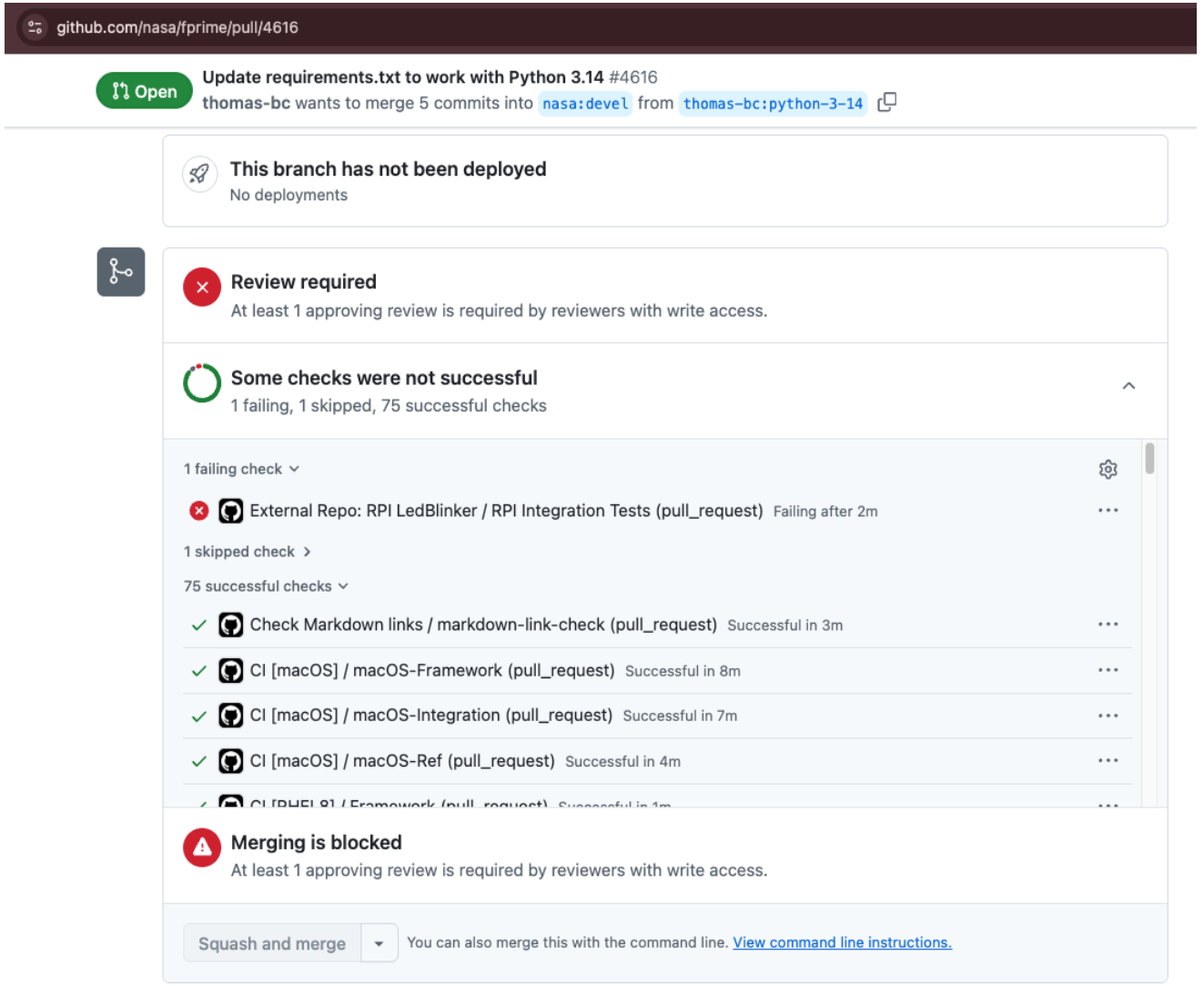
```
{
    "Svc.CommandDispatcher": "CdhCore.cmdDisp",
    "Svc.ActiveRateGroup": "ReferenceDeployment.rateGroup1Hz",
    "Svc.PrmDb": "FileHandling.prmDb",
    "Svc.PrmDb.filename": "/tmp/PrmDb.dat"
}
```

  - Validate system behavior

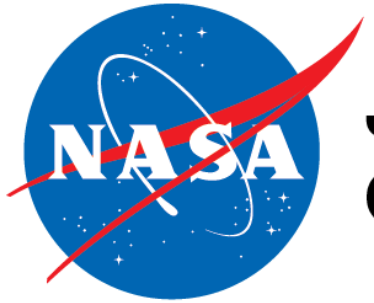
Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Continuous Integration / Continuous Deployment

- Automating builds & tests when code is submitted to the code base (open PR, commit, etc.)
- F Prime leverages GitHub Actions



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.



Jet Propulsion Laboratory
California Institute of Technology